



J D College Of Engineering & Management

Unit-3

Control Flow and Data Flow Testing

Dr. A. R. Welekar

Associate Professor(CSE)

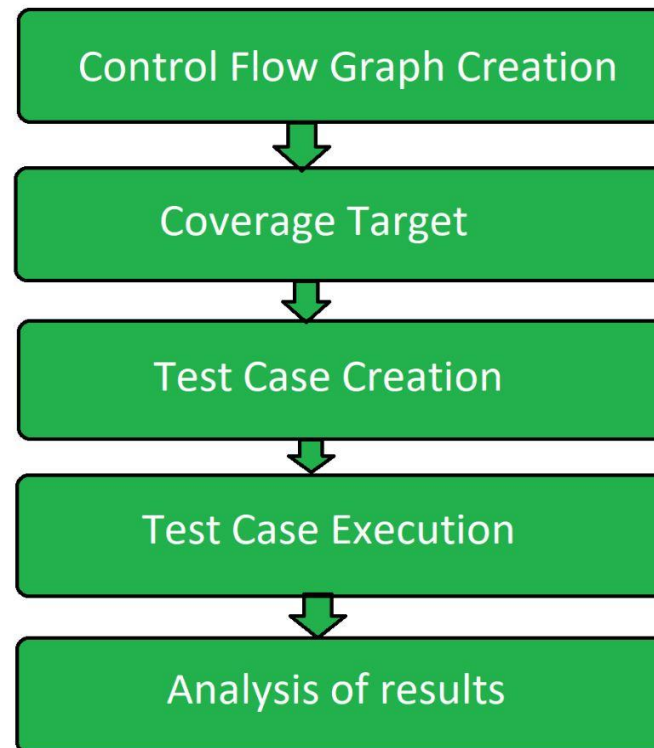
❖ Control Flow Testing:

- ❑ Control flow testing is a type of software testing that uses a program's control flow as a model. Control flow testing is a structural testing strategy. This testing technique comes under white box testing.
- ❑ For the type of control flow testing, all the structure, design, code and implementation of the software should be known to the testing team.
- ❑ This type of testing method is often used by developers to test their own code and own implementation as the design, code and the implementation is better known to the developers.
- ❑ This testing method is implemented with the intention to test the logic of the code so that the user requirements can be fulfilled. Its main application is to relate the small programs and segments of the larger programs.

❖ Control Flow Testing:

Control Flow Testing Process:

Following are the steps involved into the process of control flow testing:



❖ Control Flow Testing:

Control Flow Testing Process:

- **Control Flow Graph Creation:** From the given source code a control flow graph is created either manually or by using the software.
- **Coverage Target:** A coverage target is defined over the control flow graph that includes nodes, edges, paths, branches etc.
- **Test Case Creation:** Test cases are created using control flow graphs to cover the defined coverage target.
- **Test Case Execution:** After the creation of test cases over coverage target, further test cases are executed.
- **Analysis:** Analyze the result and find out whether the program is error free or has some defects.

❖ Control Flow Testing:

Objectives of Control Flow Testing::

- **Path Coverage:** During testing, it makes sure that every path through the program is run at least once.
- **Branch Coverage:** This guarantees that, during testing, every decision point (branch) in the program is evaluated as true or false at least once.
- **Decision Coverage:** This guarantees that all potential outcomes are covered by executing each decision point in the program at least once.
- **Loop testing:** It evaluates how the program performs in various loop scenarios, such as numerous, zero and single iterations.

❖ Control Flow Testing:

Objectives of Control Flow Testing::

- **Testing Error-Handling Paths:** It tests the program's error and exception handling pathways.
- **Multiple condition testing:** It involves testing both simple and complicated condition combinations inside decision points.
- **Boundary Value Examination:** It evaluates how the software behaves when input ranges are pushed to their limits.
- **Integration testing:** It examines how various software modules or components interact with one another.
- **Cyclomatic Complexity Reduction:** It determines and reduces the program's cyclomatic complexity.

❖ Control Flow Graphs :

- A Control Flow Graph (CFG) is the graphical representation of control flow or computation during the execution of programs or applications.
- Control flow graphs are mostly used in static analysis as well as compiler applications, as they can accurately represent the flow inside a program unit.
- The control flow graph was originally developed by Frances E. Allen.

❖ Control Flow Graphs :

Characteristics of of Control Flow Testing:

- The control flow graph is process-oriented.
- The control flow graph shows all the paths that can be traversed during a program execution.
- A control flow graph is a directed graph.
- Edges in CFG portray control flow paths and the nodes in CFG portray basic blocks.

There exist 2 designated blocks in the Control Flow Graph:

- ✓ Entry Block: The entry block allows the control to enter into the control flow graph.
- ✓ Exit Block: Control flow leaves through the exit block.

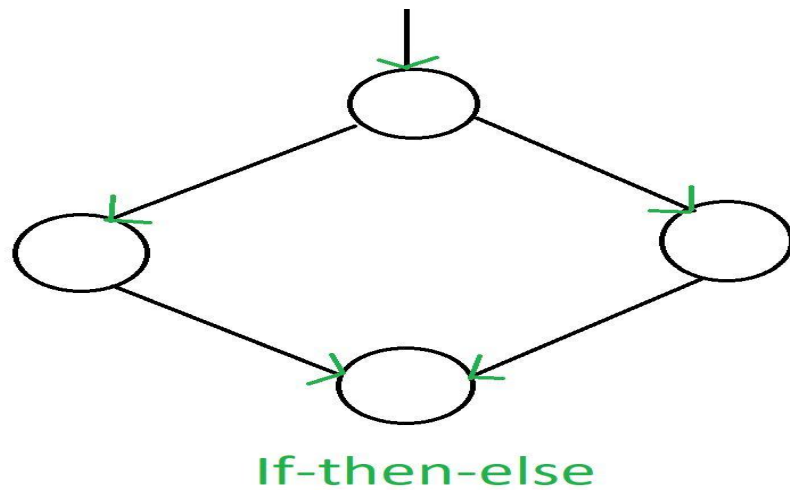
Hence, the control flow graph comprises all the building blocks involved in a flow diagram such as the start node, end node, and flows between the nodes.

❖ Control Flow Graphs :

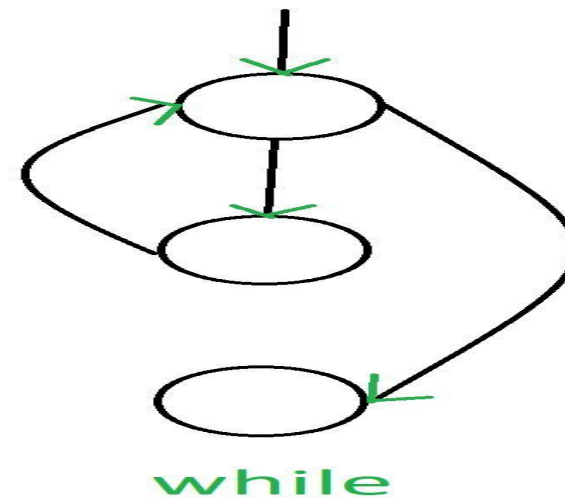
General Control Flow Graphs:

- Control Flow Graph is represented differently for all statements and loops. The following images describe it:

1. If-else



2. While

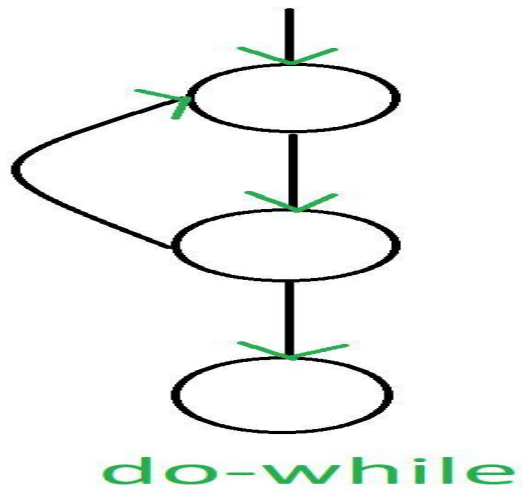


❖ Control Flow Graphs :

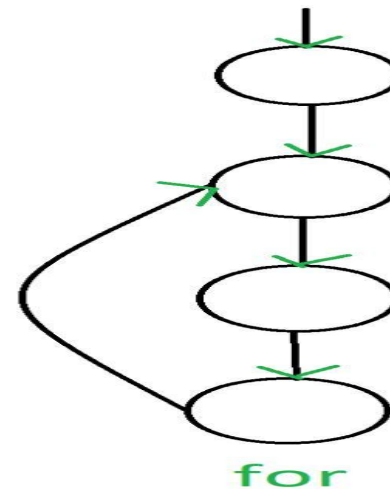
General Control Flow Graphs:

- Control Flow Graph is represented differently for all statements and loops. The following images describe it:

3. do-while

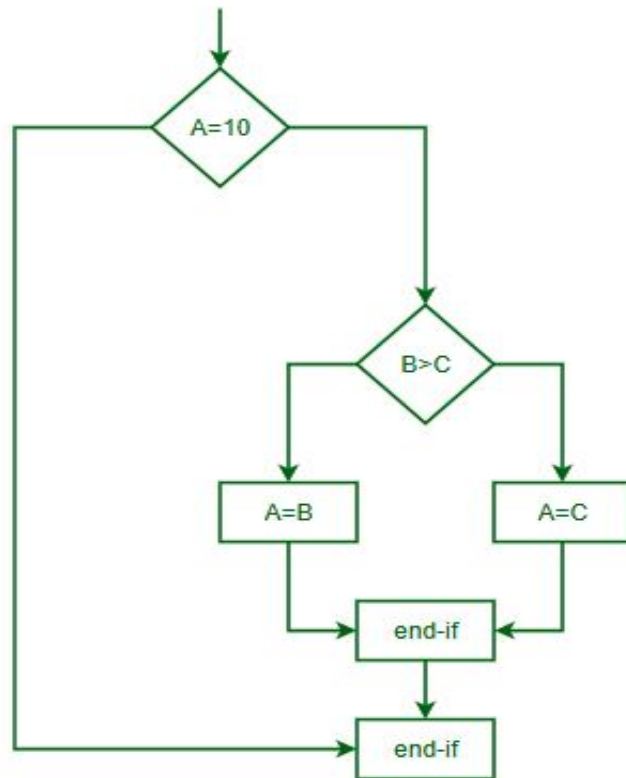


4. for

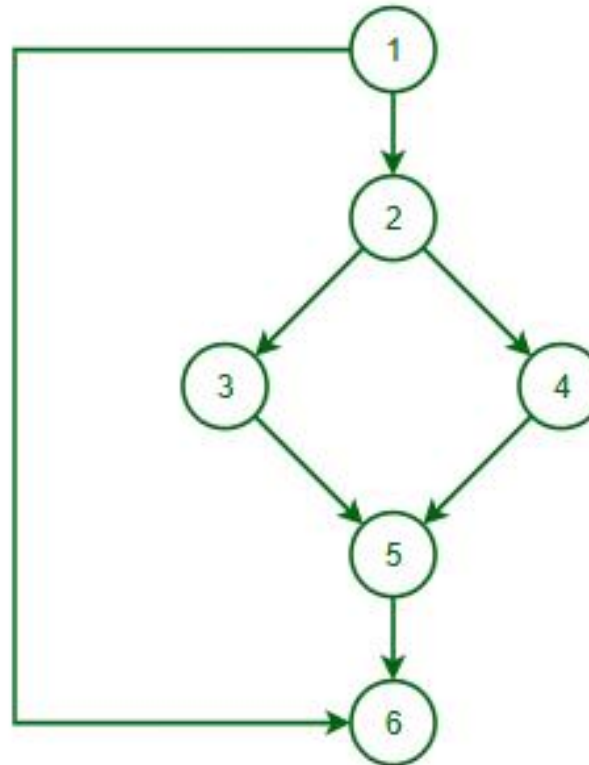


❖ Control Flow Graphs :

Flowchart of above example will be:



Control Flow Graph of above example will be:



Example:

if A = 10 then

if B > C

A = B

else A = C

endif

endif

print A, B, C

❖ Path in Control Flow Graphs :

A Path is a sequence of nodes and edges that represents one possible execution route of a program from start to end.

Types of Paths:

1. Independent Path

- A path that introduces at least one new edge not included in other paths.
- Important for Basis Path Testing

2. Feasible Path

- A path that can actually be executed with some input.

3. Infeasible Path

- A path that is logically impossible to execute.

```
1. Start
2. Read integer X
3. If (X > 10)
4.   Print "A"
5. If (X < 5)
6.   Print "B"
7. Print "End"
8. Stop
```

❖ Path Selection Criteria :

In White Box Testing, a program may have many possible execution paths. Testing all paths is usually impossible (especially with loops). So we use Path Selection Criteria to choose a set of important paths that give maximum fault detection with minimum test cases.

1. Statement Coverage:

Goal: Execute every statement at least once. 1. Rule for Path Selection:

Rule for Path Selection: Choose paths so that all nodes in the Control Flow Graph are visited.

Paths Selected:

- Path 1: $X > 0 \rightarrow$ covers both prints
- Path 2 is not required for statement coverage

```
if (X > 0)
    print("Positive");
print("Done");
```

❖ Path Selection Criteria :

2. Branch Coverage (Decision Coverage):

Goal: Execute each branch (True/False) of every decision.

Rule for Path Selection: Select paths that make each decision outcome occur at least once.

Paths Selected:

- Path 1: $X = 5 \rightarrow$ True branch
- Path 2: $X = -2 \rightarrow$ False branch

```
if (X > 0)
    print("Positive");
else
    print("Negative");
```

❖ Path Selection Criteria :

3. Condition Coverage:

Goal: Each individual condition in a decision must be True and False.

```
if (A > 10 && B < 5)  
    print("Yes");
```

Conditions:

- C1: A > 10
- C2: B < 5

Paths Selected:

A	B	C1	C2
15	3	T	T
5	3	F	T
15	8	T	F

❖ Path Selection Criteria :

4. Path Coverage:

Goal: Execute all possible paths in the program.

Possible Paths:

- $X > 0 \rightarrow A \rightarrow C$
- $X \leq 0 \rightarrow B \rightarrow C$

```
if (X > 0)
    print("A");
else
    print("B");
print("C");
```


❖ Path Selection Criteria :

5. Basis Path Testing:

This uses Cyclomatic Complexity to decide the minimum number of independent paths.

Rule for Path Selection: Select a set of independent paths that cover all edges in the CFG.

Selected Paths:

- $X > 0, Y > 0$
- $X > 0, Y \leq 0$
- $X \leq 0, Y \leq 0$

```
1. if (X > 0)
2.   print("A");
3. if (Y > 0)
4.   print("B");
5. print("End");
```

- CFG has 2 decisions
- Cyclomatic Complexity = 3
- So we select 3 independent paths

❖ Approaches for Test Data Generation:

- As a tester, your job is not just about testing the software, but also managing, collecting, and maintaining large sets of data. These data sets are crucial for testing all the major test cases to make sure the software meets all the requirements, whether it's for functional or non-functional testing.
- These data sets serve as the input for the test cases, and based on them, the software's output is generated. The behavior of the system is then analyzed to see if it matches the expected results.

❖ Approaches for Test Data Generation:

What is Test Data Generation?

- The Test Data Generation is the process of collecting and managing a large amount of data from various resources just to implement the test cases to ensure the functional soundness of the system under testing.
- These generated datasets act as the input for the test cases so that the behavior of the system can be checked.
- Test datasets are designed or selected for both positive testing or negative testing. Generating a rational and relevant dataset is a very complex task because coverage of poorly framed datasets might leave major test cases to be checked.

❖ Approaches for Test Data Generation:

Test data generation techniques:

1. Manual Test data generation:

- In this technique, all the datasets are generated manually by the tester with respect to all the required test case through experience and anticipations.
- Pros: Its easy to implement, no additional tools are needed to be deployed and Increase the confidence of the tester.
- Cons: Accuracy of data sets generated by this scheme mostly doubtful and also its time-consuming process.

❖ Approaches for Test Data Generation:

Test data generation techniques:

2. Automated Test Data Generation:

- The major feature of this testing that makes it more efficient than the above technique is the speed, automated data generation technique produces data as in an expedited manner through analyzing large volume of data in a small-time interval. In this scheme, we use automated tools, there are many available in the market.

Pros: The data sets generated by this scheme are highly accurate and Data generation speed is very fast.

Cons: The one demerit of this method is that it is a costlier method to implement and the second one is that these tools take time to understand the system.

❖ Approaches for Test Data Generation:

Test data generation techniques:

3. Back end data injection Approach:

- This method is done with the help of using SQL queries. Here a tester writes the relevant query and injects it into the database in order to populate the required data sets with respect to the test cases.
- This is also an easier method which generates a large amount of data in just a few minutes. We can update the database in this scheme if some new datasets are found through other resources like sample XML documents etc. could be updated for future use if required.

Pros: It is less time-consuming technique. Less expertise required as compared to the above technique as you only need to write a correct query to populate data required.

Cons: If you write any invalid query or incorrect it may populate illogical dataset or may cause the failure of your database system so keep attention while injecting any query into database.

❖ Approaches for Test Data Generation:

Test data generation techniques:

4. Third-party tool:

- A number of tools are available in the market that is processed or provided by the out premises tools. These tools first understand the scenarios of your system under testing and then generates dataset as per the requirement.
- These tools are customizable as per your need of the business. These tools provide wide coverage and accuracy in generating datasets.

Pros: These tools are accurate because they first understand the entire system and then generated the datasets accordingly.

Cons: Costlier technique to implement because the price of such a tool is high as compared to other technique. Less coverage in case of heterogeneous testing environment because these tools aren't generic in nature.

❖ Data Flow Testing:

Introduction:

- Data Flow Testing is a type of structural testing . It is a method that is used to find the test paths of a program according to the locations of definitions and uses of variables in the program. It has nothing to do with data flow diagrams. Furthermore, it is concerned with:
 - ✓ Statements where variables receive values,
 - ✓ Statements where these values are used or referenced.
- By analyzing control flow graphs, this technique aims to identify issues such as unused variables or incorrect definitions, ensuring proper handling of data within the code.

❖ Data Flow Testing:

Introduction:

- To illustrate the approach of data flow testing, assume that each statement in the program is assigned a unique statement number. For a statement number S-

$$\text{DEF}(S) = \{X \mid \text{statement } S \text{ contains the definition of } X\}$$

$$\text{USE}(S) = \{X \mid \text{statement } S \text{ contains the use of } X\}$$

- If a statement is a loop or if condition then its DEF set is empty and the USE set is based on the condition of statement s.

❖ Data Flow Testing:

Introduction:

- Data Flow Testing uses the control flow graph to find the situations that can interrupt the flow of the program.
- Reference or defined anomalies in the flow of the data are detected at the time of associations between values and variables. These anomalies are:
 - ✓ A variable is defined but not used or referenced,
 - ✓ A variable is used but never defined,
 - ✓ A variable is defined twice before it is used

❖ Data Flow Graph:

Introduction:

- A Data Flow Graph (DFG) is a graphical representation that shows:
 - ✓ How data moves through a program
 - ✓ Where variables are defined (assigned values)
 - ✓ Where variables are used
 - ✓ The paths connecting definitions and uses

It is mainly used in Data Flow Testing

❖ Data Flow Graph:

Basic Components of Data Flow Graph:

Component	Meaning
Node	Represents a statement or block of code
Edge	Represents flow of control
Definition (def)	Where a variable gets value
Use	Where variable is used
d-u path	Path from definition to use

There are two types of uses:

- ✓ c-use (Computational use) → Used in calculation
- ✓ p-use (Predicate use) → Used in condition

❖ Data Flow Graph:

Example:

Step 1: Identify Definitions and Uses

Line	Variable	Type
2	a	def
3	b	def
4	a, b	p-use
5	c	def
5	a, b	c-use
7	c	def
7	a, b	c-use
8	c	c-use

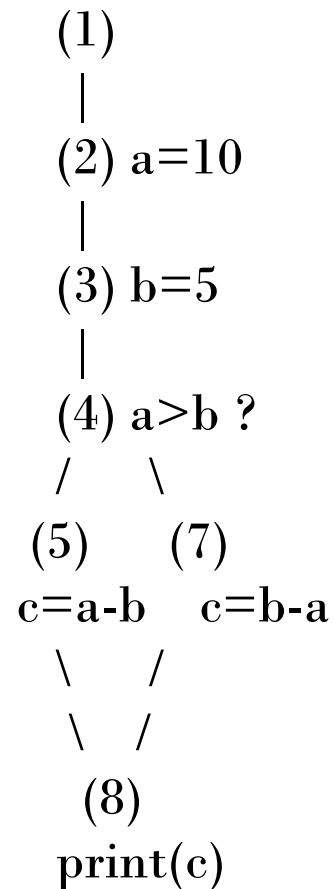
Simple Example Program

```
1. int a, b, c;  
2. a = 10;  
3. b = 5;  
4. if (a > b)  
5.   c = a - b;  
6. else  
7.   c = b - a;  
8. print(c);
```

❖ Data Flow Graph:

Simple Example Program

Step 2: Draw Data Flow Graph



```
1. int a, b, c;  
2. a = 10;  
3. b = 5;  
4. if (a > b)  
5.   c = a - b;  
6. else  
7.   c = b - a;  
8. print(c);
```

❖ Data Flow Graph:

Step 3: Data Flow Paths

Definition → Use paths for variable c

Path 1:

5 → 8

Path 2:

7 → 8

Simple Example Program

```
1. int a, b, c;  
2. a = 10;  
3. b = 5;  
4. if (a > b)  
5.   c = a - b;  
6. else  
7.   c = b - a;  
8. print(c);
```

❖ Data Flow Graph:

Why Data Flow Graph is Important:

It helps to find:

- ✓ Use of uninitialized variables
- ✓ Variables defined but not used
- ✓ Variables used but not defined
- ✓ Logical errors

Example of Error Detection

```
1. int x;  
2. int y = x + 5;
```

Here:

- ✓ x is used
- ✓ but never defined

❖ Data Flow Testing Criteria:

Data Flow Testing Criteria are the rules used to design test cases based on the definition and use of variables in a program.

These criteria ensure that every variable definition and its use is properly tested.

Main Data Flow Testing Criteria:

There are 5 important criteria:

- ✓ All Definitions (All-defs)
- ✓ All c-uses
- ✓ All p-uses
- ✓ All Uses
- ✓ All du-paths

❖ Data Flow Testing Criteria:

1. All-Definitions Criteria:

Definition: For every variable definition, at least one use must be tested

Variable	Defined at
x	Line 2
y	Line 3
z	Line 5,7

```
1. int x, y;  
2. x = 10;  
3. y = 5;  
4. if(x > y)  
5.   z = x + y;  
6. else  
7.   z = x - y;  
8. print(z);
```

❖ Data Flow Testing Criteria:

2. All c-uses Criteria:

Definition: Test all paths from definition to c-use

3. All p-uses Criteria:

Definition: Test all paths from definition to p-use

4. All Uses Criteria:

Definition: Test all c-uses and p-uses

Combination of:

- All c-uses
- All p-uses

5. All du-paths Criteria:

Definition: Test every path from definition to use

```
1. int x, y;  
2. x = 10;  
3. y = 5;  
4. if(x > y)  
5.   z = x + y;  
6. else  
7.   z = x - y;  
8. print(z);
```